Red Hat OpenShift Administration II: Configuring a Production Cluster

Course | Lab Environment

Lab: Manage Kubernetes Operators

Install an operator and verify that it is healthy.

Outcomes

- Install the compliance operator on the command line.
- Review the operator deployments and test the operator to ensure that it is installed correctly.

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start operators-review
```



Instructions

The API URL of your OpenShift cluster is `https://api.lab.example.com:6443`, and the `oc` command is already installed on your `workstation` machine.

In this exercise, you install the compliance operator. The compliance operator helps OpenShift administrators to assess the required compliance state of a cluster and provides them with an overview of gaps and remediations.

> **NOTE**
>
> For more information, refer to Compliance Operator (RHOCP 4.22) (https://docs.redhat.com/en/documentation/openshift_container_platform/4.22/html-single/security_and_compliance/index#compliance-operator).

1. Log in to the OpenShift cluster as the `admin` user with `redhatocp` as the password.

   1.1. Log in to the OpenShift cluster.

   ```
   [student@workstation ~]$ oc login -u admin -p redhatocp \
   https://api.lab.example.com:6443
   Login successful.
   ...output omitted...
   ```

   Hide Solution

2. Examine the package manifest for the compliance operator to discover the operator name, catalog name, suggested namespace, and channel.

   2.1. List the package manifest resources by running the following command:

   ```
   [student@workstation ~]$ oc get packagemanifests
   NAME                      CATALOG                  AGE
   metallb-operator          GLS Operator Catalog Cs   15d
   compliance-operator       GLS Operator Catalog Cs   15d
   lightspeed-operator       GLS Operator Catalog Cs   15d
   file-integrity-operator   GLS Operator Catalog Cs   15d
   lvms-operator             GLS Operator Catalog Cs   15d
   kubevirt-hyperconverged   GLS Operator Catalog Cs   15d
   ```

   2.2. Examine the `compliance-operator` package manifest.

```
[student@workstation ~]$ oc get packagemanifest compliance-operator -
o yaml
apiVersion: packages.operators.coreos.com/v1
kind: PackageManifest
metadata:
  creationTimestamp: "2026-08-20T10:48:23Z"
  labels:
    catalog: gls-catalog-cs
    catalog-namespace: openshift-marketplace
...output omitted...
  name: compliance-operator
  namespace: default
spec: {}
status:
...output omitted...
    channels:
  - currentCSV: compliance-operator.v1.9.2
    currentCSVDesc:
      annotations:
        alm-examples: |-
...output omitted...
        operatorframework.io/suggested-namespace: openshift-complianc
e
...output omitted...
      version: 1.9.2
    name: stable
  defaultChannel: stable
  packageName: compliance-operator
...output omitted...
```

The package manifest contains the following information:

Field	Value
catalog	gls-catalog-cs
catalog-namespace	openshift-marketplace
suggested-namespace	openshift-compliance
defaultChannel	stable
packageName	compliance-operator

Hide Solution

3. Create the recommended `openshift-compliance` namespace.

3.1. Use the `oc` command to create the namespace.

```
[student@workstation ~]$ oc create namespace openshift-compliance
namespace/openshift-compliance created
```

Hide Solution

4. Create an operator group named `compliance-operator` in the `openshift-compliance` namespace. The target namespace of the operator group is the `openshift-compliance` namespace. You can use the `~/DO280/labs/operators-review/operator-group.yaml` file as a template.

   4.1. Edit the `~/DO280/labs/operators-review/operator-group.yaml` file and modify the placeholder text to match the following content:

   ```
   apiVersion: operators.coreos.com/v1
   kind: OperatorGroup
   metadata:
     name: compliance-operator
     namespace: openshift-compliance
   spec:
     targetNamespaces:
     - openshift-compliance
   ```

   You can compare your work with the solution in the `~/DO280/solutions/operators-review/operator-group.yaml` file.

   4.2. Create the operator group by running the following command:

   ```
   [student@workstation ~]$ oc create -f \
   ~/DO280/labs/operators-review/operator-group.yaml
   operatorgroup.operators.coreos.com/compliance-operator created
   ```

   4.3. List the operator group resource in the `openshift-compliance` namespace:

   ```
   [student@workstation ~]$ oc get operatorgroup -n openshift-compliance
   NAME                  AGE
   compliance-operator   1m17s
   ```

   Hide Solution

5. Create a `compliance-operator` subscription in the `openshift-compliance` namespace. The subscription has the following parameters:

Field	Value
`channel`	`stable`

Field	Value
installPlanApproval	Automatic
spec.name	compliance-operator
source	gls-catalog-cs
sourceNamespace	openshift-marketplace

You can use the ~/DO280/labs/operators-review/subscription.yaml file as a template.

5.1. Edit the ~/DO280/labs/operators-review/subscription.yaml file and modify the placeholder text to match the following content:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: compliance-operator
  namespace: openshift-compliance
spec:
  channel: stable
  installPlanApproval: Automatic
  name: compliance-operator
  source: gls-catalog-cs
  sourceNamespace: openshift-marketplace
```

You can compare your work with the solution in the ~/DO280/solutions/operators-review/subscription.yaml file.

5.2. Create the subscription by running the following command:

```
[student@workstation ~]$ oc create -f \
~/DO280/labs/operators-review/subscription.yaml
subscription.operators.coreos.com/compliance-operator created
```

5.3. List the subscription resource in the openshift-compliance namespace:

```
[student@workstation ~]$ oc get subscription -n openshift-compliance
NAME                  PACKAGE               SOURCE           CHANNEL
compliance-operator   compliance-operator   gls-catalog-cs   stable
```

Hide Solution

6. Verify the operator installation.

The operator lifecycle manager creates a cluster service version (CSV) resource in the `openshift-compliance` namespace. Wait until the CSV resource is in the `Succeeded` phase.

Although the CSV defines a single `compliance-operator` deployment, the operator has two additional deployments. Wait until the `compliance-operator`, `ocp4-openshift-compliance-pp`, and `rhcos4-openshift-compliance-pp` deployments are ready.

6.1. Select the `openshift-compliance` project.

```
[student@workstation ~]$ oc project openshift-compliance
Now using project "openshift-compliance" on server "https://api.lab.e
xample.com:6443".
```

6.2. Run the `watch` command and wait until the compliance operator CSV is in the `Succeeded` phase.

```
[student@workstation ~]$ watch oc get csv
Every 2.0s: oc get csv                ...

NAME                         DISPLAY              VERSION   ...  PHAS
E
compliance-operator.v1.9.2   Compliance Operator  1.9.2     ...  Succ
eeded
...output omitted...
```

Press **Ctrl**+**C** to end the `watch` command after the CSV displays `Succeeded` in the `PHASE` column. The available CSV version in the lab might change. Commands in the following steps require you to replace the available version in the lab.

6.3. Inspect the CSV to view the operator deployment. Use the version that you obtained in a previous step. The `.spec.install.spec.deployments` JSONPath expression describes the location of the operator deployments in the CSV resource. Optionally, use the `jq` command to indent the output. The compliance operator describes a single deployment with the `compliance-operator` name.

```
[student@workstation ~]$ oc get csv compliance-operator.v1.9.2 \
-o jsonpath={.spec.install.spec.deployments} | jq
[
  {
    "name": "compliance-operator",
    "spec": {
...output omitted...
```

6.4. Run the `watch` command and wait until all the deployments in the `openshift-compliance` namespace are ready. In addition to the `compliance-operator` deployment, the compliance operator creates two other deployments.

```
[student@workstation ~]$ watch oc get all
Every 2.0s: oc get all                                  ...

Warning: apps.openshift.io/v1 DeploymentConfig is deprecated in v4.14
+, unavailable no sooner than v6.0+
NAME                                       ...
pod/compliance-operator-...                ...
pod/ocp4-openshift-compliance-pp-...       ...
pod/rhcos4-openshift-compliance-pp-...     ...

...output omitted...

NAME                                                READY   ...
deployment.apps/compliance-operator                 1/1     ...
deployment.apps/ocp4-openshift-compliance-pp        1/1     ...
deployment.apps/rhcos4-openshift-compliance-pp      1/1     ...

...output omitted...
```

Press **Ctrl**+**C** to end the `watch` command after the three deployments are ready.

Hide Solution

7. Verify that the compliance operator works correctly.

The compliance operator watches custom resources of the `ScanSettingBinding` type and runs compliance scans on cluster nodes. The operator reports results with custom resources of the `ComplianceSuite` type.

Create a scan setting binding in the `openshift-compliance` namespace. You can use the `~/DO280/labs/operators-review/scan-setting-binding.yaml` file that the `lab` command prepared. You do not have to change its content.

You can also use the web console to create the scan setting binding. The YAML editor in the web console provides the same scan setting binding resource as an example.

Wait until a resource of the `ComplianceSuite` type in the `DONE` phase is present in the `openshift-compliance` namespace.

7.1. Examine the `alm-examples` annotation in the CSV. Use the version that you obtained in a previous step.

```
[student@workstation ~]$ oc get csv compliance-operator.v1.9.2 \
-o jsonpath={.metadata.annotations.alm-examples} | jq
[
...output omitted...
  {
    "apiVersion": "compliance.openshift.io/v1alpha1",
    "kind": "ScanSettingBinding",
    "metadata": {
      "name": "nist-moderate"
    },
    "profiles": [
      {
        "apiGroup": "compliance.openshift.io/v1alpha1",
        "kind": "Profile",
        "name": "rhcos4-moderate"
      }
    ],
    "settingsRef": {
      "apiGroup": "compliance.openshift.io/v1alpha1",
      "kind": "ScanSetting",
      "name": "default"
    }
  },
...output omitted...
]
```

The annotation contains an example scan setting binding that you can use. The example is in JSON format. When creating a scan setting binding in the web console, the YAML editor loads the same example.

You can also use the `oc explain` command to describe the scan setting binding resource.

7.2. Create the scan setting binding resource by using the example file in the `~/DO280/labs/operators-review/scan-setting-binding.yaml` path.

```
[student@workstation ~]$ oc create \
-f ~/DO280/labs/operators-review/scan-setting-binding.yaml
scansettingbinding.compliance.openshift.io/nist-moderate created
```

7.3. Run the `watch` command and wait up to five minutes until the compliance suite resource is in the `DONE` phase. The compliance operator creates extra pods, which disappear when the scan completes.

```
[student@workstation ~]$ watch oc get compliancesuite,pod
Every 2.0s: oc get compliancesuite,pod                    ...

NAME                                                      PHASE    RESUL
T
compliancesuite.compliance.openshift.io/nist-moderate     DONE     NON-C
OMPLIANT

NAME                                                      READY    ...
pod/compliance-operator-...                               1/1      ...
pod/ocp4-openshift-compliance-pp-...                      1/1      ...
pod/rhcos4-openshift-compliance-pp-...                    1/1      ...
```

Press **Ctrl+C** to end the `watch` command after the compliance suite resource displays `DONE` in the `PHASE` column.

The compliance suite resource reports that the `nist-moderate` scan setting binding created in a previous step is `NON-COMPLIANT`, which confirms that the compliance operator is working as expected.

Hide Solution

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade operators-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish operators-review
```

Revision: do280-4.22-ed1-20260827-78c07d2



training-policies)

Terms of Use (https://www.redhat.com/en/about/terms-use)

All policies and guidelines (https://www.redhat.com/en/about/all-policies-guidelines)

Release Notes (https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Cookie Preferences and Opt-Out Rights